the splat parameter. In all cases, the splat argument receives the values left over after assigning to the positional arguments.

Hash and Keyword Parameters

Ruby allows you to define parameters to methods using keywords, with the requirement that the arguments will also be passed using the same keyword. In Calling a Method, on page 93 we'll talk more about how those arguments are passed.

The difference between a positional and keyword parameter in the method definition is that a keyword parameter name is followed by a colon:

```
def method_with_keywords(city:, state:, zip:)
end

method_with_keywords(city: "Chicago", state: "IL", zip: "60606")
```

As with positional parameters, you can specify a default value for a keyword parameter that is not called. For keyword parameters, this involves placing the default value after the colon. Keyword parameters with default values don't need be included in each call:

```
def method_with_keywords(city:, state: "IL", zip:)
end

method_with_keywords(city: "Chicago", zip: "60606")
```

When a method with keywords is called, each keyword parameter must ether be part of the call or have a default value, otherwise Ruby raises an ArgumentError.

If a method has both positional and keyword parameters, the keyword parameters must come after the positional parameters.

You can collect arbitrary keyword arguments into a Hash with the double-splat, or **:

```
def varargs(arg1, **rest)
  "arg1=#{arg1}.  rest=#{rest.inspect}"
end

varargs("one")                              # => arg1=one.  rest={}
varargs("one", color: "red")                # => arg1=one.  rest={:color=>"red"}
varargs "one", color: "red", size: "xl"     # => arg1=one.  rest={:color=>"red",
                                            # .. :size=>"xl"}
```

As with the single-splat, you can use the bare ** to ignore keyword parameters, or use it (in ruby 3.2) to pass the entire hash on to another method. A bare single splat will catch positional arguments, a bare double splat will catch keyword arguments.

```
class Child < Parent
  def do_something(**)
    do_something_else(**)
  end
end
```

Ruby also allows you to use **nil to explicitly indicate that the method does not accept any keyword arguments. Otherwise, a method definition that uses a single splat will pull in keyword arguments as a hash. If you don't want that behavior, **nil will raise an exception.